Phase 1 – Project skeleton & simulation setup

Goal: Robot running in Gazebo with a simple world and basic navigation topics.

1. **Create repo & basic structure**
   - `uncertainty-aware-safe-navigation/`
     - `ros_ws/` (catkin/colcon workspace)
     - `src/safe_nav/` (your package)
     - `worlds/` (Gazebo world)
     - `docs/` (notes, diagrams)
     - `ai_explainer/` (Python scripts for AI analysis later)
2. **Gazebo world**
   - Simple corridor / room with a few obstacles.
   - Use a standard robot (e.g. TurtleBot3) to save time.
3. **Run simulation**
   - Launch file: `sim_world.launch` (or for ROS2, a `.py` launch).
   - Verify:
     - Gazebo loads,
     - robot spawns,
     - `/cmd_vel` moves the robot via teleop.

✅ **Milestone:** "I can drive the robot around the world with teleop."

Phase 2 – Localisation with covariance

Goal: Have a pose estimate with a meaningful covariance.

1. **Add robot_localization (EKF)**
   - Configure EKF to fuse:
     - `/odom`
     - `/imu` (if available),
   - Publish: `/robot_pose_ekf` (`PoseWithCovarianceStamped`).
2. **Tune noise params**
   - Adjust process & measurement noise so covariance:
     - grows with motion / manoeuvres,
     - shrinks when sensor info is good.
3. **Visualise**
   - RViz: plot pose and (if you like) covariance ellipses.

✅ **Milestone:** "I can see the pose estimate + covariance changing as the robot moves."

Phase 3 – Global path & baseline local control

Goal: Robot follows a path without safety/uncertainty logic yet.

1. **Global path**
   - Either:
     - use Nav/NAV2 global planner, or
     - publish a custom `nav_msgs/Path` composed of waypoints.
2. **Simple local follower node**
   - New node `path_follower`:
     - Sub: `/robot_pose_ekf`, `/global_plan`
     - Output `/cmd_vel`:
       - drive along path using basic proportional controllers (distance + heading).
3. **Test**
   - Robot should move from start → goal around obstacles (if your plan avoids them).

✅ **Milestone:** "Robot can autonomously follow a path in the world."

Phase 4 – Risk-based, uncertainty-aware safety wrapper

Goal: Replace naive navigation with a **risk-aware / uncertainty-aware** local planne that enforces a safety envelope.

1. **Create `safe_local_planner` node (your main contribution)**
   - Subscriptions:
     - `/robot_pose_ekf` (pose + covariance)
     - `/scan` (LaserScan)
     - `/global_plan` (Path)
   - Publications:
     - `/cmd_vel_safe` (Twist)
     - `/risk` (Float32)
     - `/risk_state` (String: `NORMAL`, `CAUTION`, `EMERGENCY_STOP`)
2. **Uncertainty metric**
   - Extract Σ from pose covariance.
   - Compute something like `U = cov_xx + cov_yy`.
3. **Obstacle distance**
   - From `/scan`, compute min distance in a forward arc → `d_obs`.
4. **Braking distance & risk**
   - Assume max decel `a_max`.
   - `d_brake = v^2 / (2 * a_max)`.
   - `d_margin = d_obs - d_brake`.

   - Define risk:
     - `risk = w1 * max(0, -d_margin) + w2 * U`.
5. **Safety logic**
   - If `risk < R_low` → full `v_nominal`, `state = NORMAL`.
   - If `R_low ≤ risk < R_high` → reduced `v`, `state = CAUTION`.
   - If `risk ≥ R_high` → stop/creep, `state = EMERGENCY_STOP`.
6. **Integrate**
   - Send `/cmd_vel_safe` to the robot instead of bare `/cmd_vel`.

✅ **Milestone:** "Robot automatically slows/stops when uncertain or too close to obstacles."

Phase 5 – Scenarios + logging for AI

Goal: Create interesting runs and record data for analysis.

1. **Design scenarios**
   - Narrow corridor with few features → localisation uncertainty increases.
   - Sudden obstacle appears in front.
   - Sensor degradation (e.g. reduced scan FOV / increase noise).
2. **Inject issues**
   - Edit world / robot to create these situations.
   - Optionally change laser update frequency / noise.
3. **Logging**
   - Record via rosbag (or custom logger):
     - `/robot_pose_ekf`
     - `/odom`
     - `/cmd_vel_safe`
     - `/risk`, `/risk_state`
     - `/scan`
   - Convert to CSV for AI (e.g. a Python script in `ai_explainer/`).

✅ **Milestone:** "I have rosbag/CSV logs of normal & stressed runs with risk, uncertainty and events."

Phase 6 – AI "Safety Explainer"

Goal: AI assistant that turns logs + your safety rules into human-readable insights.

1. **Prepare a small RAG knowledge base**
   - In `ai_explainer/docs/`, create a few markdown files:
     - `uncertainty.md` – why localisation uncertainty matters.
     - `risk_metric.md` – explanation of your risk formula.
     - `braking_distance.md` – concept of stopping distance & margin.
     - `safety_principles.md` – your own notes (slow under uncertainty, fa safe, etc.).
2. **Log processing script**
   - Python script (e.g. `analyze_run.py`) that:
     - loads CSV,
     - finds:
       - peaks in `risk`,
       - transitions in `risk_state`,
       - large changes in speed.
     - builds text snippets: e.g.
       - "From 10–14s: speed ↓ from 0.5→0.1 m/s, U ↑, d_obs ↓…"
3. **LLM + RAG use**
   - Feed each snippet + relevant docs to an LLM (even if you only *conceptuall* hook it up / show examples).
   - Generate:
     - explanations of why the robot slowed/stopped,
     - suggestions like "lower `R_high` in narrow corridors" or "consider sens redundancy".
4. **Save outputs**
   - Store AI summaries as `.md` report(s) per run, e.g. `reports/run1_safety_analysis.md`.

✅ **Milestone:** "Given a run, I can produce an AI-generated safety analysis explaini risky moments and system behaviour."

Final polish – for the meeting / GitHub

- **README.md**
  - 1–2 sentence project summary.
  - System diagram (image).
  - How to run sim + planner.
  - Short example of an AI-generated explanation.
- **Demo plan**
  - One launch file: `safe_nav_demo.launch`.
  - One log file + AI analysis shown as markdown or notebook.
- **Slides (optional)**
  - 5–7 slides following: problem → architecture → behaviour → AI explainer → relevance to functional safety.